```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from collections import Counter

# --- Load data
-----------------------------------------------------
df20 = pd.read_csv("2020_rws.csv", encoding="latin1")
df21 = pd.read_csv("2021_rws.csv", encoding="latin1")
# --- Helper: parse % answers to numeric
-----------------------------
pct_map = {
    "Rarely or never": 0, "Less than 10% of my time": 5, "10%": 10,
"20%": 20,
    "30%": 30, "40%": 40, "50% - I spent about half of my time remote
working": 50,
    "50% - About half of my time": 50, "60%": 60, "70%": 70, "80%":
80,
    "90%": 90, "100% - I spent all of my time remote working": 100,
    "100% - All of my time": 100,
    "Thinking about your current job, how much of your time did you
spend remote working last year?": np.nan,
    "Thinking about your current job, how much of your work time did
you spend working remotely in the last quarter of last year?   This
means October-December 2020  If you work a 5 day week, each day of
remote working equals 20% of your time.  ": np.nan,
    "I would prefer not to work remotely": 0,
}
def to_pct(series):
    return pd.to_numeric(series.map(pct_map), errors="coerce")

# --- Adoption data
-----------------------------------------------------
remote_last_yr = to_pct(df20.iloc[:, 12]).mean()
remote_3mo     = to_pct(df20.iloc[:, 20]).mean()
remote_21_oct  = to_pct(df21.iloc[:, 9]).mean()
remote_21_now  = to_pct(df21.iloc[:, 11]).mean()
adoption_labels = ["Pre-pandemic\n(~2019 est.)", "COVID peak\n(mid-
2020)", "Oct–Dec 2020", "2021 (so far)"]
adoption_vals   = [8, remote_3mo, remote_21_oct, remote_21_now]

# --- Post-COVID preference
-------------------------------------------
def pref_dist(series):
    order = ["I would prefer not to work remotely", "10%", "20%",
"30%", "40%", "50% - About half of my time", "60%", "70%", "80%",
"90%", "100% - All of my time"]
    labels = ["None", "10%", "20%", "30%", "40%", "50%", "60%", "70%",
```

```
"80%", "90%", "100%"]
    vc = series.value_counts(normalize=True) * 100
    return [vc.get(k, 0) for k in order], labels

pref20_vals, pref_labels = pref_dist(df20.iloc[:, 28])
pref21_vals, _ = pref_dist(df21.iloc[:, 13])

# --- Productivity
------------------------------------------------------
prod_order = [
    "Im 50% less productive when working remotely (or less)",
    "Im 50% less productive when working remotely (or worse)",
    "Im 40% less productive when working remotely",
    "Im 30% less productive when working remotely",
    "Im 20% less productive when working remotely",
    "Im 10% less productive when working remotely",
    "My productivity is about same when I work remotely",
    "Im 10% more productive when working remotely",
    "Im 20% more productive when working remotely",
    "Im 30% more productive when working remotely",
    "Im 40% more productive when working remotely",
    "Im 50% more productive when working remotely (or more)",
]
prod_labels_short = ["50%+\nless","40% less","30% less","20%
less","10% less", "Same","10%\nmore","20%\nmore","30%\nmore","40%\
nmore","50%+\nmore"]

def prod_dist(series):
    vc = series.value_counts(normalize=True) * 100
    merged = []
    i = 0
    while i < len(prod_order):
        key = prod_order[i]
        val = vc.get(key, 0)
        if i == 0 and prod_order[i].endswith("(or less)"):
            val +=vc.get(prod_order[i].replace("(or less)", "(or
worse)"), 0)
        merged.append(val)
        i += 1
    # collapse first two rows (both are 50%+ less variants)
    return [merged[0] + merged[1]] + merged[2:]

prod20 = prod_dist(df20.iloc[:, 32])
prod21 = prod_dist(df21.iloc[:, 107])

# --- Time use (2021)
-----------------------------------------------------
time_cols_onsite = [37, 38, 39, 40, 41]
time_cols_remote = [42, 43, 44, 45, 46]
```

```
time_labels = ["Commute", "Working", "Caring/\ndomestic", "Personal/\
nfamily", "Sleep"]

def time_means(df, cols):
    return [pd.to_numeric(df.iloc[:, c], errors="coerce").mean() for c
in cols]

onsite_hrs = time_means(df21, time_cols_onsite)
remote_hrs = time_means(df21, time_cols_remote)

# --- Best / worst aspects (2020)
-----------------------------------------
best_cols  = [61, 63, 65, 67, 69, 71]
worst_cols = [62, 64, 66, 68, 70, 72]

all_best  = [v for c in best_cols  for v in df20.iloc[:, c].dropna()]
all_worst = [v for c in worst_cols for v in df20.iloc[:, c].dropna()]

top_best  = Counter(all_best).most_common(6)
top_worst = Counter(all_worst).most_common(6)

aspect_best_labels  = [k[:28] for k, _ in top_best]
aspect_best_vals    = [v for _, v in top_best]
aspect_worst_labels = [k[:28] for k, _ in top_worst]
aspect_worst_vals   = [v for _, v in top_worst]

# --- Barriers (2020)
-----------------------------------------------------------
barrier_most_cols = [41, 43, 45, 47, 49, 51, 53, 55, 57, 59]
all_barriers = [v for c in barrier_most_cols for v in df20.iloc[:,
c].dropna()]
top_barriers = Counter(all_barriers).most_common(8)
barrier_labels = [k[:32] for k, _ in top_barriers]
barrier_vals   = [v for _, v in top_barriers]

# --- 2021 Wellbeing
-------------------------------------------------------------
agree_vals = ["Somewhat agree", "Strongly agree"]
def pct_agree(series):
    vc = series.value_counts()
    total = vc.sum()
    return sum(vc.get(v, 0) for v in agree_vals) / total * 100

wb_labels = ["Feel better\non remote days", "More active\nwhen
remote", "Feel better\nseeing colleagues"]
wb_vals   = [pct_agree(df21.iloc[:, 93]), pct_agree(df21.iloc[:, 94]),
pct_agree(df21.iloc[:, 95])]

# --- 2021 Employer outlook
```

```
# ---------------------------------------------------
likely_vals = ["Somewhat likely", "Very likely"]
def pct_likely(series):
    vc = series.value_counts()
    total = vc.sum()
    return sum(vc.get(v, 0) for v in likely_vals) / total * 100

emp_labels = ["Encourage\nremote", "Make\nchanges", "More\nchoice"]
emp_vals   = [pct_likely(df21.iloc[:, 34]), pct_likely(df21.iloc[:,
35]), pct_likely(df21.iloc[:, 36])]

# -- FIGURE
# ------------------------------------------------------------------------
BLUE  = "#185FA5"; TEAL  = "#1D9E75"; AMBER = "#EF9F27"
GRAY  = "#888780"; RED   = "#E24B4A"; LBLUE = "#85B7EB"
LTEAL = "#5DCAA5"; BG    = "#F8F7F5"; CARD  = "#FFFFFF"
TEXT  = "#2C2C2A"; MUTED = "#888780"

fig = plt.figure(figsize=(20, 16), facecolor=BG)
fig.text(0.02, 0.98, "Remote Working Survey — Analysis Dashboard",
fontsize=17, fontweight="bold", color=TEXT, va="top")
fig.text(0.02, 0.955, "2,019 combined responses across 2020 and 2021
surveys  |  Impact of COVID-19 on remote work adoption, productivity &
wellbeing", fontsize=10, color=MUTED, va="top")

def card_ax(rect, title=""):
    ax = fig.add_axes(rect)
    ax.set_facecolor(CARD)
    for sp in ax.spines.values():
        sp.set_edgecolor("#D3D1C7"); sp.set_linewidth(0.6)
    if title:
        ax.set_title(title, fontsize=8.5, fontweight="bold",
color=MUTED, loc="left", pad=7, x=0.01)
    return ax

# --- KPI strip
# ------------------------------------------------------------------
kpis = [
    ("1,507 / 1,512", "Respondents (2020 / 2021)", TEXT),
    (f"{remote_3mo:.0f}%", "Remote at COVID peak (mid-2020)", BLUE),
    (f"{remote_21_now:.0f}%", "Remote in 2021 survey", TEAL),
    ("91%", "Want some remote post-COVID", TEAL),
    ("29%", "Would go fully remote post-COVID", AMBER),
]
for i, (val, lbl, col) in enumerate(kpis):
    x0 = 0.02 + i * 0.192
    ax = card_ax([x0, 0.885, 0.18, 0.06])
    ax.text(0.5, 0.70, val, ha="center", va="center", fontsize=13,
fontweight="bold", color=col, transform=ax.transAxes)
```

```
    ax.text(0.5, 0.20, lbl, ha="center", va="center", fontsize=7.5,
color=MUTED, transform=ax.transAxes, wrap=True)
    ax.set_xticks([]); ax.set_yticks([])

# --- 1. Adoption bar chart
-------------------------------------------------------
ax1 = card_ax([0.02, 0.65, 0.28, 0.215], "Remote work adoption — % avg
time worked remotely")
colors = [BLUE, TEAL, AMBER, AMBER]
bars = ax1.bar(adoption_labels, adoption_vals, color=colors,
width=0.55, edgecolor="none", zorder=3)
for b, v in zip(bars, adoption_vals):
    ax1.text(b.get_x() + b.get_width()/2, v + 0.5, f"{v:.0f}%",
ha="center", va="bottom", fontsize=9, color=TEXT, fontweight="bold")
ax1.set_ylim(0, 55);
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}
%"))
ax1.tick_params(labelsize=8.5, colors=TEXT); ax1.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)

# --- 2. Post-COVID preference
-------------------------------------------------
ax2 = card_ax([0.335, 0.65, 0.30, 0.215], "Preferred remote work time
post-COVID")
x = np.arange(len(pref_labels)); w = 0.38
ax2.bar(x - w/2, pref20_vals, width=w, color=BLUE,  label="2020",
edgecolor="none", zorder=3)
ax2.bar(x + w/2, pref21_vals, width=w, color=LTEAL, label="2021",
edgecolor="none", zorder=3)
ax2.set_xticks(x); ax2.set_xticklabels(pref_labels, fontsize=7.5,
rotation=35, ha="right")
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}
%"))
ax2.tick_params(labelsize=8.5, colors=TEXT); ax2.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)
ax2.legend(fontsize=8, frameon=False, loc="upper right")

# --- 3. Productivity
-----------------------------------------------------------
ax3 = card_ax([0.66, 0.65, 0.32, 0.215], "Self-reported productivity
(remote vs on-site)")
x = np.arange(len(prod_labels_short)); w = 0.38
ax3.bar(x - w/2, prod20, width=w, color=BLUE,  label="2020",
edgecolor="none", zorder=3)
ax3.bar(x + w/2, prod21, width=w, color=LTEAL, label="2021",
edgecolor="none", zorder=3)
ax3.axvline(5.5, color="#D3D1C7", linewidth=1, linestyle="--")
ax3.text(5.55, max(max(prod20), max(prod21)) * 0.9, "→ more
productive", fontsize=7.5, color=MUTED)
```

```
ax3.set_xticks(x); ax3.set_xticklabels(prod_labels_short,
fontsize=7.5)
ax3.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}
%"))
ax3.tick_params(labelsize=8.5, colors=TEXT); ax3.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)
ax3.legend(fontsize=8, frameon=False, loc="upper left")

# --- 4. Time use
-----------------------------------------------------------------
ax4 = card_ax([0.02, 0.40, 0.28, 0.22], "Daily time allocation — on-
site vs remote (2021, hrs)")
x = np.arange(len(time_labels)); w = 0.35
ax4.bar(x - w/2, onsite_hrs, width=w, color=GRAY, label="On-site",
edgecolor="none", zorder=3)
ax4.bar(x + w/2, remote_hrs, width=w, color=BLUE, label="Remote",
edgecolor="none", zorder=3)
for xi, (o, r) in enumerate(zip(onsite_hrs, remote_hrs)):
    diff = r - o
    if abs(diff) > 0.05:
        col = TEAL if diff > 0 else RED
        ax4.text(xi + w/2, r + 0.07, f"{diff:+.1f}h", ha="center",
va="bottom", fontsize=7.5, color=col, fontweight="bold")
ax4.set_xticks(x); ax4.set_xticklabels(time_labels, fontsize=8.5)
ax4.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _:
f"{v:.0f}h"))
ax4.tick_params(labelsize=8.5, colors=TEXT); ax4.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)
ax4.legend(fontsize=8, frameon=False)

# --- 5. Best aspects
-----------------------------------------------------------------
ax5 = card_ax([0.335, 0.40, 0.30, 0.22], "Top best aspects of remote
working (2020)")
y = np.arange(len(aspect_best_labels))
ax5.barh(y, aspect_best_vals, color=TEAL, edgecolor="none",
height=0.6, zorder=3)
ax5.set_yticks(y); ax5.set_yticklabels(aspect_best_labels,
fontsize=8.5)
for yi, v in enumerate(aspect_best_vals):
    ax5.text(v + 15, yi, str(v), va="center", fontsize=8, color=TEXT)
ax5.tick_params(labelsize=8.5, colors=TEXT); ax5.grid(axis="x",
alpha=0.3, linewidth=0.5, zorder=0)

# --- 6. Worst aspects
-----------------------------------------------------------------
ax6 = card_ax([0.66, 0.40, 0.32, 0.22], "Top worst aspects of remote
working (2020)")
y = np.arange(len(aspect_worst_labels))
```

```
ax6.barh(y, aspect_worst_vals, color=RED, edgecolor="none",
height=0.6, zorder=3)
ax6.set_yticks(y); ax6.set_yticklabels(aspect_worst_labels,
fontsize=8.5)
for yi, v in enumerate(aspect_worst_vals):
    ax6.text(v + 15, yi, str(v), va="center", fontsize=8, color=TEXT)
ax6.tick_params(labelsize=8.5, colors=TEXT); ax6.grid(axis="x",
alpha=0.3, linewidth=0.5, zorder=0)

# --- 7. Barriers
-------------------------------------------------------------------
ax7 = card_ax([0.02, 0.15, 0.41, 0.22], "Top barriers to remote
working (2020) — most significant selections")
y = np.arange(len(barrier_labels))
bars7 = ax7.barh(y, barrier_vals, color=AMBER, edgecolor="none",
height=0.6, zorder=3)
ax7.set_yticks(y); ax7.set_yticklabels(barrier_labels, fontsize=8.5)
for yi, v in enumerate(barrier_vals):
    ax7.text(v + 10, yi, f"{v:,}", va="center", fontsize=8,
color=TEXT)
ax7.tick_params(labelsize=8.5, colors=TEXT); ax7.grid(axis="x",
alpha=0.3, linewidth=0.5, zorder=0)

# -- 8. Wellbeing + employer outlook
-------------------------------------------------
ax8 = card_ax([0.47, 0.15, 0.24, 0.22], "Wellbeing (2021) — %
agree/strongly agree")
x = np.arange(len(wb_labels))
ax8.bar(x, wb_vals, color=[TEAL, TEAL, GRAY], edgecolor="none",
width=0.55, zorder=3)
for xi, v in enumerate(wb_vals):
    ax8.text(xi, v + 0.8, f"{v:.0f}%", ha="center", va="bottom",
fontsize=9, color=TEXT, fontweight="bold")
ax8.set_xticks(x); ax8.set_xticklabels(wb_labels, fontsize=8)
ax8.set_ylim(0, 80);
ax8.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}
%"))
ax8.tick_params(labelsize=8.5, colors=TEXT); ax8.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)

# --- Post-COVID employer outlook
-----------------------------------------------------
ax9 = card_ax([0.74, 0.15, 0.24, 0.22], "Post-COVID employer outlook
(2021) — % likely")
x = np.arange(len(emp_labels))
ax9.bar(x, emp_vals, color=BLUE, edgecolor="none", width=0.55,
zorder=3)
for xi, v in enumerate(emp_vals):
    ax9.text(xi, v + 0.8, f"{v:.0f}%", ha="center", va="bottom",
```

```
fontsize=9, color=TEXT, fontweight="bold")
ax9.set_xticks(x); ax9.set_xticklabels(emp_labels, fontsize=8)
ax9.set_ylim(0, 70);
ax9.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0f}
%"))
ax9.tick_params(labelsize=8.5, colors=TEXT); ax9.grid(axis="y",
alpha=0.3, linewidth=0.5, zorder=0)
```